

Technical Documentation

Ahmed Fawaizat Oyindamola (fawah595)
Abdul Gaffar Abubakar Passum (abuab954)
Varun Gurupurandar (vargu125)
Mehran Mamivand (mehma172)

February 11, 2026

Version 1.0



Status

Reviewed	Firstname Lastname	date
Approved	Firstname Lastname	date

Project Identity

Group E-mail: tsks23-group3@liu.se

Orderer & Customer: Danyo Danev
Phone: +46 (0)13 28 13 35
E-mail: danyo.danev@liu.se

Supervisor: Oksana Moryakova
Phone: +46 (0)13-28 15 53
E-mail: oksana.moryakova@liu.se

Supervisor: David Nordlund
Phone: +46 (0)13-28 19 53
E-mail: david.nordlund@liu.se

Supervisor: Henrik Åkesson
Phone: +46 (0)13-28 28 95
E-mail: henrik.akesson@liu.se

Course Responsible: Danyo Danev
Phone: +46 (0)13-28 13 35
E-mail: danyo.danev@liu.se

Participants of the group

Name	Role	E-mail
Fawaizat Ahmed	Project Lead	fawah595@student.liu.se
Abdul Gaffar Abubakar Passum	Design Lead	abuab954@student.liu.se
Varun Gurupurandar	Test Lead	vargul25@student.liu.se
Mehran Mamivand	Documentation Lead	mehma172@student.liu.se

CONTENTS

1	Introduction	1
2	Previous Work and Technical Background	1
3	System Overview	1
3.1	System Functionality	1
3.2	System Architecture	2
3.3	Operational Workflow	2
4	Graphical User Interface	4
4.1	Design Rationale and Technology Choice	4
4.2	GUI Architecture Overview	4
4.2.1	Main Application Window	4
4.2.2	File Selection and Audio Loading	5
4.2.3	Live Audio Recording	5
4.2.4	Transcription Settings Control	5
4.2.5	Visualization Panel	5
4.2.6	Transcription Results and Confidence Statistics	6
4.2.7	Export Functionality	6
5	Backend Processing and Operational Workflow	7
5.1	Overview	7
5.2	Audio Input and Initial Representation	7
5.2.1	Connection to GUI and Database	7
5.3	Audio Preprocessing	7
5.4	Onset and Offset Detection	7
5.4.1	STFT and Spectral Flux Onset Strength Envelope	9
5.4.2	Offset Estimation	9
5.5	Pitch Detection	10
5.5.1	Monophonic Pitch Detection	10
5.5.2	Polyphonic Pitch Detection	11
5.6	Note Mapping, Integration, and Post-Processing	11
5.7	Summary	11
6	Database Management System	11
6.1	Database Initialization and Schema Creation	12
6.2	Audio File Management	12
6.3	Transcription Sessions	12
6.4	Event-Level Representation	13
6.5	Individual Note Storage	13
6.6	Confidence Statistics	13
6.7	Data Retrieval and GUI Integration	13
6.8	Relational Terminology	14
6.9	Design Summary	14
7	Results Summary	14
7.1	Onset and Offset Detection	14
7.2	Monophonic Pitch Estimation	14
7.3	Polyphonic Pitch Estimation	14
7.4	Summary Table	15
8	Limitations and Future Work	15
8.1	Limitations	15
9	Conclusion and Future Work	15
9.1	Future Work	15

DOCUMENT HISTORY

Version	Date	Changes made	Sign	Reviewer
0.1	2025-12-15	Initial technical documentation draft based on implemented system	AF, AG, VG, MM	DN
0.2	2025-12-23	Updated GUI walkthrough, export steps, troubleshooting	AF, AG, VG, MM	DN
0.3	2025-12-25	Documentation first submission draft	AF, AG, VG, MM	DN
1.0	2026-02-04	Final draft	AF, AG, VG, MM	DN

1 INTRODUCTION

This document constitutes the technical documentation for the *Automatic Music Transcription System*, developed as part of the course **TSKS23 – Project Course in Signal Processing, Communications and Networking (CDIO)** at **Linköping University**. The project was carried out during the 2025 academic year by Group 3.

The purpose of this document is to provide a detailed technical description of the system architecture, design decisions, and implementation details. It is intended to be sufficiently comprehensive to enable an external reader, possessing the same level of prior knowledge as the project group at the start of the course, to understand the system and continue its development without additional guidance.

This documentation explains how the approved requirements are realised in practice and focuses on the internal structure of the system, including its modular decomposition, signal processing pipeline, software components, and interfaces between subsystems. Emphasis is placed on design rationale and technical workflows rather than end-user operation.

For a complete understanding of how to install and operate the system from a user perspective, this document should be read in conjunction with the *User Manual*, which describes usage scenarios and interaction with the graphical user interface.

2 PREVIOUS WORK AND TECHNICAL BACKGROUND

This project builds on established techniques in automatic music transcription (AMT), particularly time-domain pitch estimation and time–frequency analysis [1].

For monophonic pitch estimation, the YIN algorithm [2] was selected due to its robustness against octave errors and its suitability for practical pitch estimation in music signals. YIN is widely used in speech and music applications and provides reliable fundamental frequency estimates for isolated note segments.

For polyphonic transcription, time–frequency representations based on the Constant-Q Transform (CQT) were used. The logarithmic frequency spacing of the CQT aligns well with musical pitch perception and enables detection of multiple simultaneous notes.

Classical autocorrelation and related variants such as AMDF are foundational approaches in pitch estimation [3], [4]. The purpose of this section is to provide technical context for the chosen methods rather than a comprehensive survey. Detailed mathematical formulations and comparative evaluations are outside the scope of this technical documentation and are not required to understand or reproduce the implemented system.

3 SYSTEM OVERVIEW

This section provides a high-level overview of the Automatic Music Transcription System, describing its objectives, operating principles, and overall architecture. The system is designed to convert acoustic piano audio into structured musical information by automatically detecting note onsets, offsets, pitch, and duration for both monophonic and polyphonic input signals.

The implementation distinguishes between monophonic and polyphonic transcription tasks and employs different signal processing techniques accordingly. Monophonic pitch estimation is performed using the YIN algorithm, while polyphonic analysis relies on time-frequency representations derived from the Constant-Q Transform (CQT). This hybrid approach balances accuracy and computational efficiency while addressing the inherent challenges of polyphonic music transcription. Below is a block diagram of the entire system

3.1 System Functionality

At a functional level, the system performs the following tasks:

- Acquisition of audio signals from pre-recorded files or real-time recording sources.
- Preprocessing of the audio signal to reduce noise and normalise amplitude.
- Detection of note onsets and estimation of corresponding offsets.

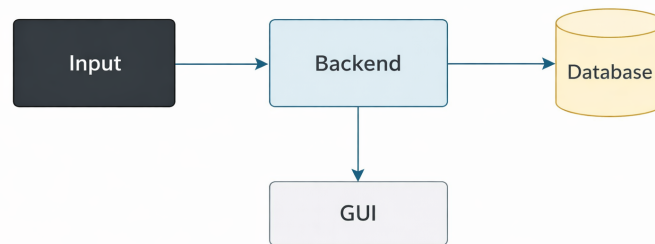


Figure 1: High-level block diagram of the Automatic Music Transcription System showing the interaction between input acquisition, backend processing, graphical user interface, and database storage.

- Monophonic pitch estimation using the YIN fundamental frequency estimator.
- Polyphonic pitch analysis using Constant-Q Transform-based spectral representations.
- Mapping of detected frequencies to musical note names and durations.
- Presentation, storage, and export of transcription results.

The processing path is selected dynamically based on the selected transcription mode and the characteristics of the input signal.

3.2 System Architecture

The system follows a modular architecture composed of five main subsystems:

1. **Input Handling System**, responsible for audio acquisition and buffering.
2. **Preprocessing Module**, which conditions the audio signal for analysis.
3. **Core Processing Engine**, implementing onset detection, YIN-based monophonic pitch estimation, and CQT-based polyphonic analysis.
4. **Output and Visualisation Module**, which presents transcription results through textual and graphical representations.
5. **Data Management Module**, which handles persistent storage and retrieval of transcription data.

Each subsystem communicates through clearly defined interfaces, enabling independent development and facilitating future extensions such as alternative pitch estimators or additional instrument support.

3.3 Operational Workflow

Figure 2 illustrates the overall workflow of the system. Audio data are first acquired and preprocessed before being forwarded to the core processing engine. Following onset detection, the signal is analysed using either the YIN algorithm for monophonic transcription or CQT-based methods for polyphonic transcription. The resulting pitch and timing information is then converted into symbolic musical representations and presented to the user or stored for later retrieval.

This structured workflow ensures a clear separation between signal acquisition, analysis, and presentation, improving maintainability, reproducibility, and extensibility of the system.

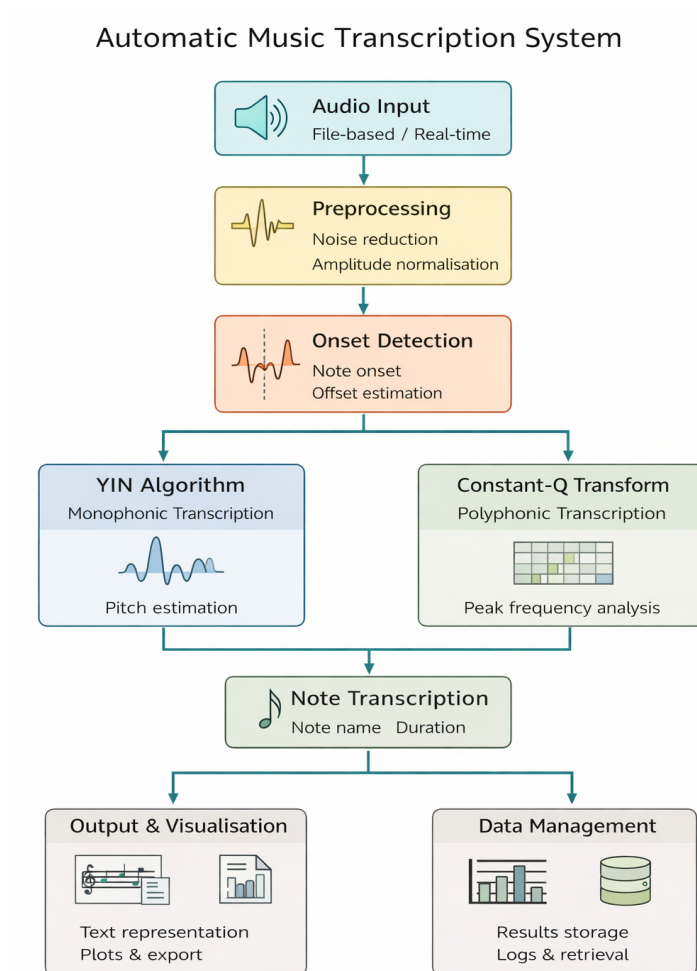


Figure 2: High-level system overview of the Automatic Music Transcription System. The pipeline includes audio acquisition, preprocessing, onset detection, and separate analysis paths for monophonic transcription using the YIN algorithm and polyphonic transcription using the Constant-Q Transform.

4 GRAPHICAL USER INTERFACE

4.1 Design Rationale and Technology Choice

The Graphical User Interface (GUI) serves as the primary interaction layer between the user and the Automatic Music Transcription System. The GUI was implemented using **PyQt5**, a Python binding for the Qt framework, chosen for its robustness, cross-platform support, and seamless integration with Python-based signal processing libraries.

PyQt5 enables the construction of complex, event-driven desktop applications while maintaining responsiveness during computationally intensive backend operations. Its signal-slot architecture allows safe communication between user interface elements and backend processing modules, which is essential for real-time audio recording, visualization updates, and transcription feedback.

The GUI integrates **Matplotlib** using the `Qt5Agg` backend to provide high-quality, interactive waveform and spectrogram visualizations directly within the application window. This allows users to visually inspect audio signals and transcription results without leaving the interface.

4.2 GUI Architecture Overview

The GUI is implemented in the file `gui.py` and is structured around two main classes:

- `AudioProcessorGUI (QMainWindow)`
- `AudioPlayerWidget (QWidget)`

The application follows a modular layout, separating user interaction, visualization, transcription control, and result presentation. The interface is divided into two main panels using a `QSplitter`:

- **Left panel:** Input controls, transcription settings, results, and confidence statistics
- **Right panel:** Visualization tabs for waveform and spectrogram display

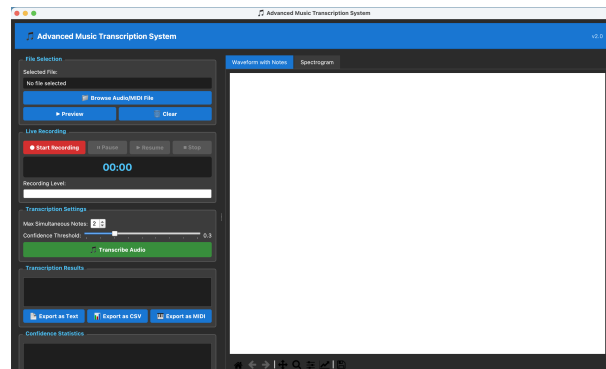


Figure 3: Main graphical user interface of the Automatic Music Transcription System, showing input controls, transcription settings, visualisation area, and result panels.

This separation ensures clarity and scalability of the interface.

4.2.1 Main Application Window

The `AudioProcessorGUI` class inherits from `QMainWindow` and acts as the central controller for all GUI operations. It initializes and coordinates the following backend components:

```
self.audio_processor = AudioProcessor(sample_rate=44100)
self.transcription_engine = TranscriptionEngine(sample_rate=44100)
self.recording_manager = RecordingManager()
self.visualization_engine = VisualizationEngine()
```

This design ensures that user interface logic remains decoupled from signal processing, improving maintainability and testability.

4.2.2 File Selection and Audio Loading

The *File Selection* group allows users to load audio (.wav, .mp3, .flac) or MIDI files. The method

```
browse_file()
```

invokes a `QFileDialog`, validates the selected file type, and delegates loading to backend utility functions:

- `load_audio_file()` for audio signals
- `transcribe_midi_file()` for MIDI input

Once loaded, metadata such as duration and sampling rate are displayed, and the waveform visualization is automatically generated via

```
self.plot_waveform()
```

4.2.3 Live Audio Recording

Live recording functionality is implemented through the `RecordingManager` class and controlled via the GUI buttons:

- `start_recording()`
- `pause_recording()`
- `resume_recording()`
- `stop_recording()`

A `QTimer` periodically updates the recording duration display and input level meter. Upon stopping a recording, the captured audio is automatically passed to the transcription engine, enabling a seamless record-and-transcribe workflow.

4.2.4 Transcription Settings Control

The *Transcription Settings* panel enables runtime configuration of transcription parameters, including:

- Maximum simultaneous notes (`QSpinBox`)
- Confidence threshold (`QSlider`)

Changes to these controls directly affect backend processing through:

```
max_notes = self.note_spinbox.value()
confidence_threshold = self.confidence_slider.value() / 100.0
```

This allows real-time experimentation with both monophonic and polyphonic transcription behavior.

4.2.5 Visualization Panel

The visualization panel consists of a `QTabWidget` with two tabs:

- Waveform with Notes
- Spectrogram

Each tab embeds a `Matplotlib FigureCanvas` with navigation tools. Visualization functions include:

- `plot_waveform()`
- `plot_waveform_with_notes()`
- `plot_spectrogram()`

Detected onsets, offsets, note regions, and confidence values are overlaid using color-coded markers and annotations, providing immediate visual feedback of transcription accuracy.

4.2.6 *Transcription Results and Confidence Statistics*

Transcription results are presented in a structured textual format via `QTextEdit`. Each detected event includes onset and offset times, pitch values, note labels, and confidence indicators.

Confidence statistics are computed using

```
calculate_confidence_stats()
```

and summarized in a dedicated panel, enabling users to evaluate transcription reliability quantitatively.

4.2.7 *Export Functionality*

The GUI supports exporting transcription results to multiple formats:

- Text (`.txt`)
- CSV (`.csv`)
- MIDI (`.mid`)

Export operations are handled by

```
export_results()  
export_to_midi()
```

This enables integration with external tools such as digital audio workstations and music notation software.

5 BACKEND PROCESSING AND OPERATIONAL WORKFLOW

5.1 Overview

The backend constitutes the core processing engine of the Automatic Music Transcription System. It is responsible for transforming raw audio input into structured symbolic musical information. The backend is implemented in the module `audio_processor.py` and follows a modular object-oriented design in which each class encapsulates a specific signal processing task.

This design enables clear separation of concerns, simplifies debugging, and allows individual processing stages to be evaluated and visualised independently while remaining tightly integrated with the graphical user interface and database.

The backend pipeline is orchestrated by the `TranscriptionEngine` class and consists of the following stages:

- Audio input handling and preprocessing
- Onset and offset detection
- Pitch detection (monophonic and polyphonic)
- Post-processing, confidence filtering, and event merging

5.2 Audio Input and Initial Representation

Audio input is handled by the `AudioProcessor` class and originates either from pre-recorded audio files selected via the GUI or from real-time recordings captured by the `RecordingManager`.

The function

```
load_audio()
```

loads the input signal, converts it to a monophonic waveform, and resamples it to a fixed sampling rate of 44.1 kHz to ensure consistent temporal and spectral resolution throughout the pipeline.

Figure 4 shows an example piano recording prior to preprocessing.

5.2.1 Connection to GUI and Database

The GUI triggers audio loading or recording via PyQt5 signal-slot connections. Metadata such as file name, sampling rate, duration, and timestamps are persisted in the database to support session reconstruction and reproducibility.

5.3 Audio Preprocessing

Audio preprocessing is implemented in the `AudioProcessor` class and prepares the input signal for robust onset detection and pitch estimation. This stage reduces sensitivity to recording conditions and suppresses low-frequency artefacts that negatively affect spectral analysis.

The preprocessing pipeline consists of amplitude normalisation followed by high-pass filtering. Normalisation compensates for variations in recording gain, while high-pass filtering attenuates low-frequency noise and handling artefacts. The filtered signal is used in all subsequent backend stages.

Figure 5 illustrates the waveform and STFT spectrogram after preprocessing.

5.4 Onset and Offset Detection

Onset and offset detection is implemented in the `EnhancedOnsetDetector` class. The goal of this stage is to segment the input signal into note events that can be processed independently for pitch estimation.

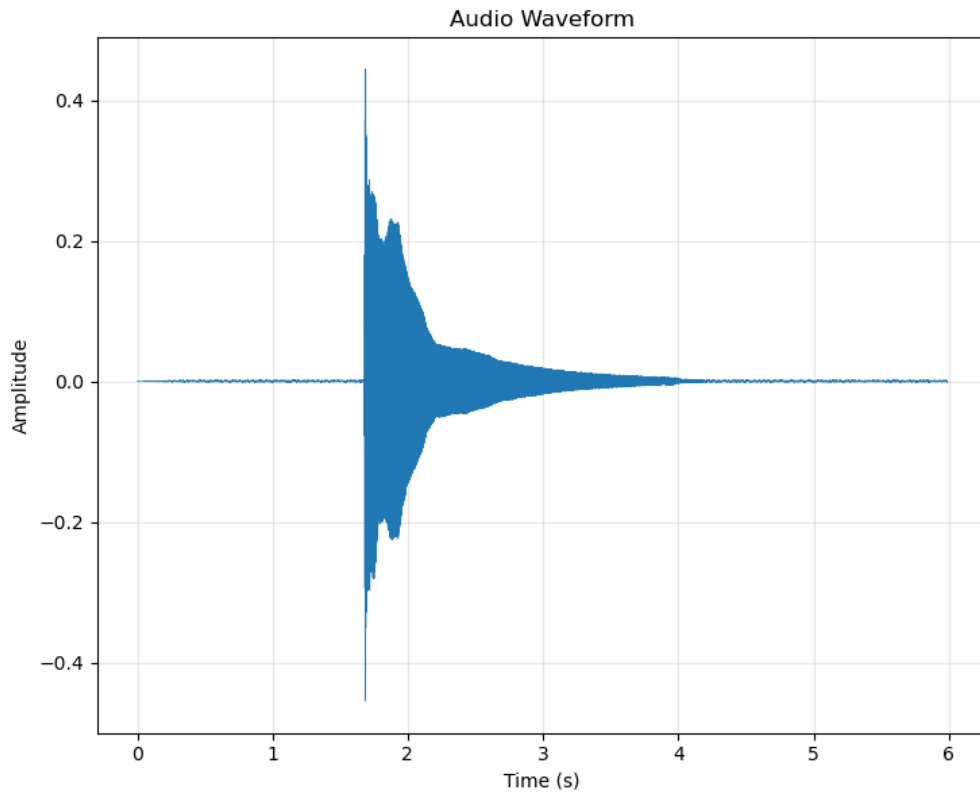


Figure 4: Time-domain waveform of the input audio signal before preprocessing.

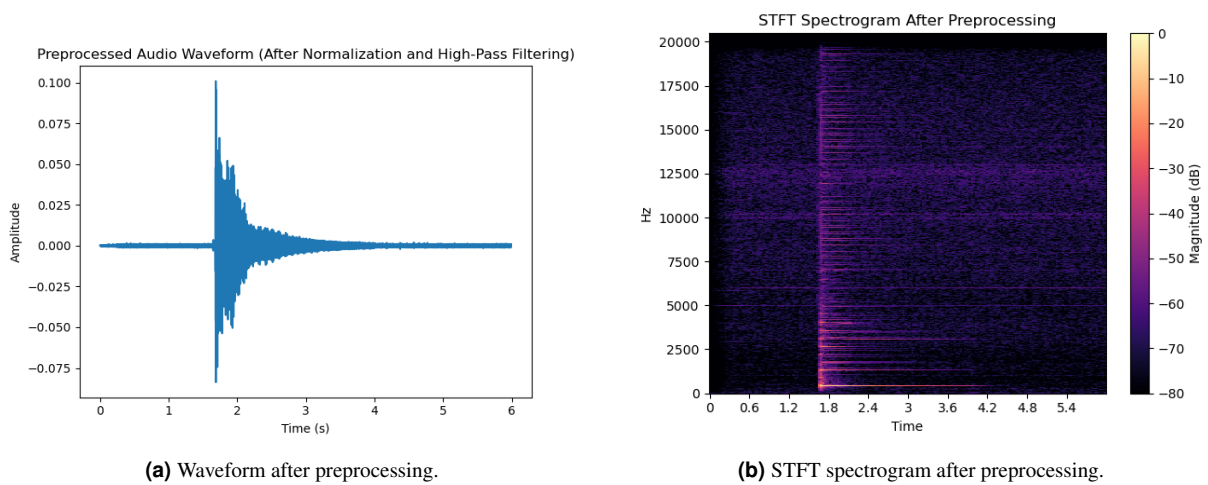


Figure 5: Effect of preprocessing on the time-domain signal and its time-frequency representation.

5.4.1 STFT and Spectral Flux Onset Strength Envelope

Let $x[n]$ be the preprocessed audio signal sampled at rate f_s . The signal is framed using a hop size H and an analysis window of length N . The Short-Time Fourier Transform (STFT) is computed for each frame n :

$$X(k, n) = \sum_{m=0}^{N-1} x[m + nH] w[m] e^{-j2\pi km/N} \quad (1)$$

where k is the frequency-bin index and $w[m]$ is the analysis window. The magnitude spectrum is $|X(k, n)|$.

The onset strength envelope is computed using spectral flux, i.e., the positive change in spectral magnitude between consecutive frames:

$$SF(n) = \sum_k \max(0, |X(k, n)| - |X(k, n-1)|) \quad (2)$$

A smoothing filter (moving average) is applied to $SF(n)$ to reduce spurious peaks. Peaks above an adaptive threshold are selected as onset candidates. Backtracking is used to refine the onset location by searching backwards from each peak to the nearest preceding local minimum in the onset envelope.

Detected onset frame indices n_i are converted to time instants as:

$$t_{on,i} = \frac{n_i H}{f_s} \quad (3)$$

5.4.2 Offset Estimation

Offset estimation is performed in the backend by the function `estimate_offsets(audio_data, onsets, min_duration)` in the `EnhancedOnsetDetector` class. :contentReference[oaicite:1]index=1 Given detected onset times $\{t_i\}_{i=1}^N$, the goal is to select a corresponding offset time $t_{off,i}$ for each note event, while preventing unrealistically short segments and avoiding overlap with the next onset.

Let $x[n]$ be the discrete-time audio signal sampled at f_s Hz, and let $n_i = \lfloor t_i f_s \rfloor$ be the onset sample index. For each onset t_i , the search region for the offset is limited to the interval from n_i up to the next onset (or the end of the signal):

$$n_{\max,i} = \begin{cases} \lfloor t_{i+1} f_s \rfloor, & i < N, \\ N_x, & i = N, \end{cases} \quad (4)$$

where N_x is the total number of samples.

ENVELOPE COMPUTATION (HILBERT AMPLITUDE ENVELOPE). To obtain a smooth representation of the note decay, the implementation computes the analytic signal via the Hilbert transform and uses its magnitude as an amplitude envelope:

$$z[n] = x[n] + j \mathcal{H}\{x[n]\}, \quad a[n] = |z[n]|. \quad (5)$$

The envelope is then smoothed using a Savitzky–Golay filter (20 ms window, polynomial order 3) and normalized to $[0, 1]$:

$$\tilde{a}[n] = \frac{a_{\text{smooth}}[n]}{\max_n a_{\text{smooth}}[n]}. \quad (6)$$

This corresponds to the code path using `hilbert()`, `savgol_filter()`, and normalization.

PRIMARY OFFSET CRITERION (SUSTAINED DECAY BELOW A LOW THRESHOLD). For each onset, the algorithm finds a local peak within the first 100 ms after onset and sets a very low decay threshold based on that peak:

$$\theta_i = \max(0.015, 0.03 \cdot \tilde{a}[n_{\text{peak},i}]). \quad (7)$$

Offset search begins after the attack phase (50 ms after onset). The method scans forward (taking sparse samples for efficiency) and selects the first time where the envelope remains below θ_i for a sustained period. When this sustained-low condition is met, the selected offset is backtracked slightly to the last point where the envelope was still above approximately $2\theta_i$, to better align with the perceived note end.

FALLBACK 1 (SETTLING/LOW-VARIANCE WAVEFORM CRITERION). If the envelope criterion does not yield an offset, a fallback checks whether the waveform has “settled” by scanning short windows (30 ms) and selecting the first window where both the mean absolute amplitude and the waveform variance are sufficiently small, and remain small in the subsequent window. This provides a second, time-domain criterion for detecting the end of the note tail.

FALLBACK 2 (RMS ENERGY DECAY). If neither envelope nor settling criteria produce an offset, the algorithm computes a framewise RMS envelope (frame length 2048, hop 512), smooths it, and selects the first time where the RMS energy decays below approximately 10% of the local post-onset peak and remains low for several frames.

FALLBACK 3 (BOUNDED BY NEXT ONSET). If no offset is found by the above methods, the offset is set close to the next onset (leaving a small gap), i.e., near the end of the available search interval.

ENERGY-COVERAGE ADJUSTMENT (CAPTURE THE NOTE TAIL). After an initial offset candidate $\hat{n}_{\text{off},i}$ is obtained, the implementation ensures that the selected segment captures most of the note energy in the region:

$$u_i[n] = |x[n_i + n]|, \quad n = 0, \dots, (\hat{n}_{\text{off},i} - n_i), \quad (8)$$

$$c_i[n] = \sum_{m=0}^n u_i[m]. \quad (9)$$

The algorithm finds the earliest index n_{95} such that $c_i[n_{95}] \geq 0.95 c_i[\hat{n}_{\text{off},i} - n_i]$, and if n_{95} would extend the offset further, it updates the offset to capture at least 95% of the cumulative absolute-amplitude energy in the note region.

POST-CONSTRAINTS (MINIMUM DURATION, NON-OVERLAP, PADDING). Finally, the offset is constrained to ensure musically meaningful segments and prevent overlap:

- **Minimum duration:** the duration is enforced to be at least $\max(\text{min_duration}, 0.2)$ seconds.
- **Non-overlap:** if there is a next onset t_{i+1} , the offset is limited to be at least 30 ms before t_{i+1} .
- **Safety padding:** an additional 50 ms is added to the selected offset when possible, to capture more of the decay tail.
- **Sanity check:** if the resulting offset would not be after the onset, a default short duration is enforced.

These steps match the final constraint logic in the implementation.

This multi-stage procedure explains why the earlier draft mentioned multiple candidate offset times: the implemented algorithm explicitly generates candidates from an envelope-based method, two fallbacks, and a final bounded default, and then applies energy coverage and timing constraints to obtain the final offset used by the GUI and demo.

Figure 6 shows detected onsets and offsets overlaid on the waveform.

5.5 Pitch Detection

Pitch detection is implemented in the `EnhancedPitchDetector` class. The behaviour of this class adapts dynamically based on the number of simultaneous notes requested via the GUI.

5.5.1 Monophonic Pitch Detection

For monophonic segments, pitch estimation is performed using the YIN algorithm [2] via:

```
detect_pitch_yin()
```

Intuition: YIN estimates the fundamental period by measuring how well the waveform matches a time-shifted version of itself. When the shift τ equals the true period, the difference between the two signals becomes small. YIN improves classical autocorrelation by using a normalized difference function and selecting the first reliable minimum, which reduces octave errors.

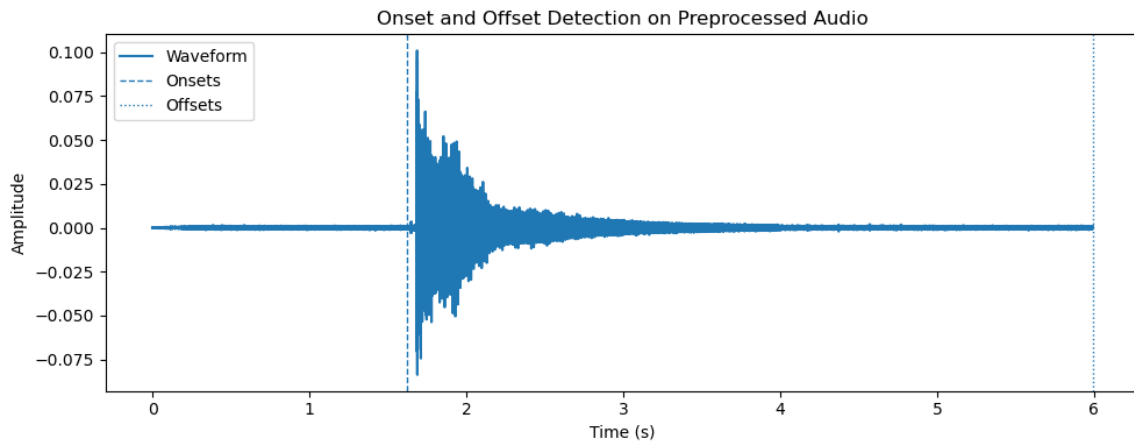


Figure 6: Detected onsets and offsets overlaid on the preprocessed waveform.

In the implementation, pitch candidates are computed for each note segment and filtered using a confidence criterion. Low-confidence estimates are discarded before mapping the remaining pitch estimate to a musical note label.

5.5.2 Polyphonic Pitch Detection

For polyphonic transcription, the function

```
detect_pitches_cqt()
```

computes a Constant-Q Transform to obtain a logarithmically spaced time–frequency representation aligned with musical pitch perception. Spectral peaks are extracted, ranked by magnitude, and mapped to pitch candidates.

The maximum number of simultaneous detected pitches is constrained by GUI parameters to reduce false positives and improve interpretability.

5.6 Note Mapping, Integration, and Post-Processing

Detected frequencies are converted into musical note labels using the `NoteMapping` class, which implements equal-tempered tuning with $A4 = 440$ Hz.

The `TranscriptionEngine` class coordinates all backend components via

```
transcribe_audio()
```

This method integrates onset detection, pitch estimation, confidence filtering, and segment merging to produce musically meaningful transcription events. Final results are forwarded to the GUI and stored in the database.

5.7 Summary

The backend processing engine combines a modular software architecture with well-established signal processing techniques. Each class corresponds to a clearly defined stage in the transcription pipeline, enabling transparency, extensibility, and robust performance for both monophonic and polyphonic music transcription.

6 DATABASE MANAGEMENT SYSTEM

The system employs a lightweight relational database implemented using **SQLite** to persist audio metadata, transcription results, confidence measures, and session-level statistics. All database operations are encapsulated in the `DatabaseManager` class implemented in `database_manager.py`.

The database is initialized automatically at application startup and is tightly integrated with the backend processing engine and the graphical user interface.

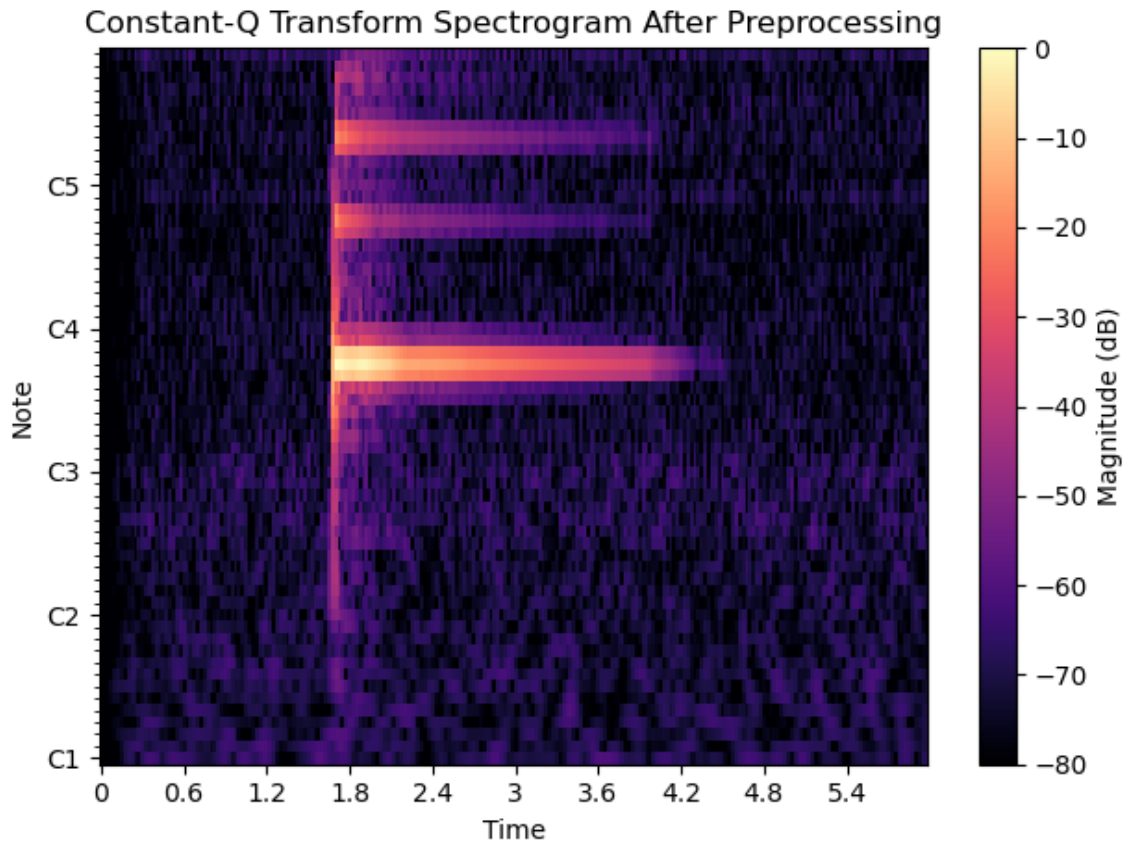


Figure 7: Constant-Q Transform spectrogram used for polyphonic pitch analysis.

6.1 Database Initialization and Schema Creation

Database creation and schema initialization are handled by the function

```
init_database()
```

This function creates all required tables if they do not already exist and defines the relational structure used throughout the system. The schema is designed to support session-based transcription, polyphonic note representation, and confidence-based analysis.

Figure 8 illustrates the resulting database schema.

6.2 Audio File Management

Audio-level metadata is managed through the `audio_files` table. Each entry represents a unique audio source, either a file-based input or a recorded session.

Audio file records are created before transcription and referenced by all subsequent processing stages. This design allows multiple transcription sessions to be associated with the same audio input.

6.3 Transcription Sessions

Each execution of the transcription pipeline is stored as a separate session using the function

```
save_transcription_session()
```

Session-level information includes the selected detection method, confidence threshold, total number of detected events and notes, and processing time. This enables reproducibility and comparison of different transcription configurations applied to the same audio signal.

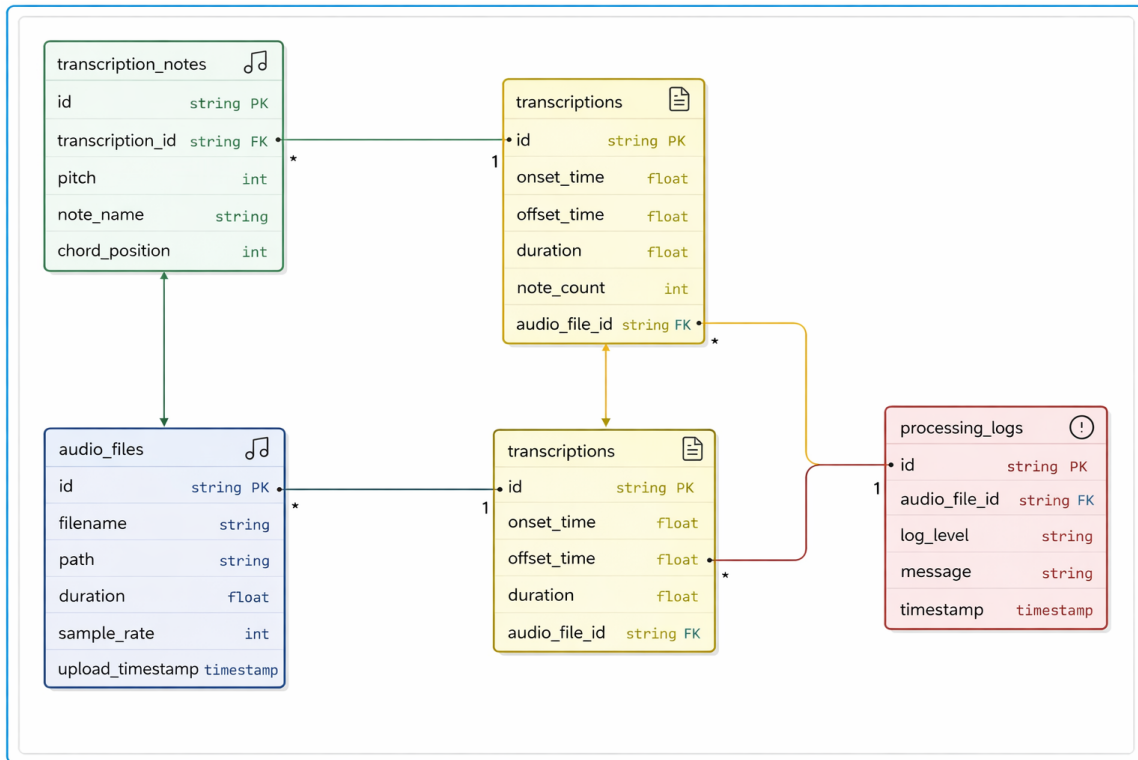


Figure 8: Relational database schema used by the system.

The `transcription_sessions` table acts as the central linking entity between audio files and note-level transcription data.

6.4 Event-Level Representation

Detected note events are stored in the `note_events` table. Each event corresponds to a time interval defined by an onset and offset and may represent either a monophonic note or a polyphonic chord.

Event-level confidence values are computed during insertion and represent the mean confidence of the associated individual notes. Event ordering is preserved using onset time.

6.5 Individual Note Storage

Individual pitch estimates are stored in the `individual_notes` table. Each entry represents a single detected note and contains pitch information, confidence value, detection method (YIN or CQT), and relative ordering within a chord.

This table enables one-to-many relationships between note events and individual pitches, which is essential for representing polyphonic transcription results.

6.6 Confidence Statistics

Aggregate confidence metrics are computed automatically during session persistence and stored using the `confidence_stats` table. These statistics include measures of central tendency and counts of high- and low-confidence detections.

Confidence statistics are used by the GUI to present quantitative summaries of transcription reliability and support threshold-based performance evaluation.

6.7 Data Retrieval and GUI Integration

Historical transcription data is retrieved using the functions

```
get_session_history()  
get_session_details()
```

These functions are invoked by the GUI to display previous transcription sessions, inspect note events and pitch details, and support export of transcription results. All database interactions are read-only at this stage, ensuring data integrity.

6.8 Relational Terminology

Table 1 summarizes key relational database concepts used in the schema.

Table 1: Relational database terminology used in the system.

Term	Description
Primary Key	Unique identifier for each table entry
Foreign Key	Reference linking entries across tables
One-to-Many	Relationship between events and individual notes
Session-Based Schema	Separation of audio inputs and transcription runs
Referential Integrity	Enforcement of valid cross-table references

6.9 Design Summary

The database schema mirrors the logical structure of the transcription pipeline rather than raw signal processing output. By organizing data around audio inputs, transcription sessions, event-level timing, and note-level pitch estimates, the system achieves clarity, scalability, and traceability while supporting both monophonic and polyphonic transcription workflows.

7 RESULTS SUMMARY

This section summarizes the key observed results for the final system configuration. Detailed verification procedures, pass/fail criteria, and requirement-level validation are documented in the Test Protocol.

7.1 Onset and Offset Detection

Onset detection was evaluated using piano recordings spanning low, mid, and high pitch ranges. In controlled test conditions, the mean absolute onset timing error was observed to be below 50 ms, satisfying the performance requirement defined in the requirements specification.

Offset estimation was based on onset-to-onset segmentation with a minimum duration constraint. During the demo, offset behaviour was more sensitive than onset detection, particularly for notes with long decay. A revised demo will be provided to confirm stable offset behaviour in the final configuration.

7.2 Monophonic Pitch Estimation

Monophonic pitch estimation was evaluated on isolated piano notes. The system achieved 100% correct note-label detection for the tested monophonic set, consistent with the values reported in the Test Protocol.

7.3 Polyphonic Pitch Estimation

Polyphonic pitch estimation was evaluated on two-note combinations using the CQT-based detection path. Across the evaluated dataset, polyphonic pitch detection accuracy was observed in the range 70%–85%, depending on the chosen confidence threshold. Higher thresholds reduced false detections but could suppress weaker pitch components.

7.4 Summary Table

Table 2: Summary of key performance results for the final system configuration.

Task	Metric	Observed performance
Onset detection	Mean absolute error	≤ 50 ms
Monophonic pitch detection	Note-label accuracy	100% (tested set)
Polyphonic pitch detection (2 notes)	Accuracy	70%–85%

8 LIMITATIONS AND FUTURE WORK

8.1 Limitations

The system is optimized for piano-like tones and performs best on monophonic or lightly polyphonic recordings. Polyphonic transcription remains challenging due to overlapping harmonics and spectral masking. Offset estimation is based on segmentation heuristics and may differ from natural decay in expressive playing.

9 CONCLUSION AND FUTURE WORK

This project presented the design, implementation, and evaluation of an Automatic Music Transcription System capable of performing onset and offset detection, as well as pitch detection for both monophonic and lightly polyphonic piano recordings. The system integrates signal processing techniques with a modular backend architecture, a PyQt5-based graphical user interface, and a relational SQLite database.

In controlled test conditions, the system demonstrated reliable monophonic transcription and stable onset/offset segmentation. Polyphonic transcription performance was competitive within the defined project scope, while remaining sensitive to confidence threshold settings and the inherent challenges of overlapping harmonics.

Detailed quantitative results, pass/fail criteria, and validation evidence are reported in the Test Protocol.

9.1 Future Work

Several directions for future improvement have been identified:

- Enhancing polyphonic pitch detection through advanced harmonic grouping or machine learning-based approaches.
- Incorporating adaptive or note-dependent confidence thresholds to improve detection robustness.
- Extending the system to support additional instruments and musical styles.
- Improving offset estimation by modeling note decay characteristics.
- Evaluating system performance under real-world recording conditions and noisy environments.
- Optimizing database and backend components to support larger datasets and real-time operation.

Overall, the project demonstrates that accurate and interpretable automatic music transcription can be achieved using classical signal processing techniques, while providing a solid foundation for future extensions and research-oriented improvements.

REFERENCES

- [1] E. Benetos, S. Dixon, D. Giannoulis, H. Kirchhoff, and A. Klapuri, “Automatic music transcription: Challenges and future directions,” *Journal of Intelligent Information Systems*, vol. 41, pp. 407–434, 2013.

- [2] A. de Cheveigné and H. Kawahara, “Yin, a fundamental frequency estimator for speech and music,” *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.
- [3] L. R. Rabiner, “On the use of autocorrelation analysis for pitch detection,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 1, pp. 24–33, 1977.
- [4] M. J. Ross, H. L. Shaffer, A. Cohen, R. Freudberg, and H. J. Manley, “Average magnitude difference function pitch extractor,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 22, no. 5, pp. 353–362, 1974.